

Guia de Estudo Completo - Segunda Prova de Algoritmos e Estruturas de Dados

Disciplina: Algoritmos e Estrutura de Dados

Professor: Marcelo Santos Linder

Data da Segunda Prova: 15 de outubro de 2025

Estudante: João Emanuel Almeida Ramos

Resumo Executivo

Com base na ementa da disciplina e na análise da playlist de videoaulas do Prof. Marcelo Linder, foram identificados **11 vídeos essenciais** para a preparação da segunda prova, totalizando **5 horas e 9 minutos** de conteúdo. A prova aborda três grandes temas: **Filas**, **Pilhas** e **Árvores**, com ênfase em implementações estáticas e dinâmicas.

Tópicos da Segunda Prova (conforme cronograma):

- Filas: conceitos, implementação estática sequencial e encadeada dinâmica
- Pilhas: conceitos, implementação estática sequencial e dinâmica encadeada
- Árvores: conceitos gerais, árvores binárias (alocação estática e dinâmica)
- Árvores Binárias de Busca: definições e operações
- Árvores Balanceadas: Árvore AVL

I. FILAS (Queue) - FIFO (First In, First Out)

1.1 Conceitos Fundamentais

Como o Professor Linder explica: "Uma fila é uma lista com restrições de acesso, onde seguimos o critério FIFO - first in, first out. O primeiro item que entra é o primeiro a sair da estrutura."

Operações Básicas:

- **Enqueue (Enfileirar):** inserção no final da fila
- **Dequeue (Desenfileirar):** remoção do início da fila
- **Front:** acesso ao elemento da frente sem removê-lo
- **isEmpty:** verificação se a fila está vazia

1.2 Implementação Estática Sequencial

▮ Videoaula 18 - Disciplinas de Acesso - Fila (32 min)

Link: https://www.youtube.com/watch?v=vZbFTtr--QU&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=18

Estrutura de Dados:

```
#define MAX 100

typedef struct {
    int VAL[MAX];    // Vetor para armazenar elementos
    int INICIO;      // Cursor para início da fila
    int FIM;         // Cursor para fim da fila
    int N;           // Número de elementos na fila
} FILASEQ;
```

Explicação do Professor: "Utilizamos dois cursores - INICIO e FIM - para controlar a fila. O truque está em fazer o cursor de inserção assumir 0 quando chegar a MAX-1, usando o operador módulo."

Operações Implementadas:

1. Criar Fila:

```
void criafila(FILASEQ *f) {
    f->INICIO = 0;
    f->FIM = -1;
    f->N = 0;
}
```

Explicação: Inicializa os cursores e o contador. FIM começa em -1 porque será incrementado na primeira inserção.

2. Verificar se está vazia:

```
int ehvazia(FILASEQ *f) {
    return (f->N == 0);
}
```

Explicação: Testa o contador N para determinar se a fila está vazia.

3. Inserir elemento (Enqueue):

```
void ins(FILASEQ *f, int v) {
    if (f->N == MAX) {
        printf("Fila cheia!");
        exit(1);
    }
    f->FIM = (f->FIM + 1) % MAX; // Fila circular
    f->VAL[f->FIM] = v;
    f->N++;
}
```

Explicação: O operador % MAX implementa o conceito de fila circular, reaproveitando posições vazias no início.

4. Consultar primeiro elemento:

```
int cons(FILASEQ *f) {
    if (ehvazia(f)) {
        printf("Fila vazia!");
        exit(1);
    }
    return f->VAL[f->INICIO];
}
```

5. Remover elemento (Dequeue):

```
void ret(FILASEQ *f) {
    if (ehvazia(f)) {
        printf("Fila vazia!");
        exit(1);
    }
    f->INICIO = (f->INICIO + 1) % MAX;
    f->N--;
}
```

Explicação: Incrementa INICIO circularmente e decrementa o contador.

1.3 Implementação Dinâmica Encadeada

▮ Videoaula 19 - Fila Encadeada (27 min)

Link: https://www.youtube.com/watch?v=Gq9V6wzyrVY&list=PLMx6FZj7-18bH9UWzsY31zf45eHnOI_dO&index=19

Estruturas de Dados:

```
typedef struct nodo {
    int info;
    struct nodo* prox;
} NODO;

typedef struct {
    NODO* INICIO;    // Ponteiro para primeiro nó
    NODO* FIM;       // Ponteiro para último nó
} FILAENC;
```

Explicação do Professor: "Utilizamos um descritor com duas referências - ao primeiro e ao último nodo - para facilitar o acesso a ambas as extremidades da lista."

Operações Implementadas:

1. Criar Fila:

```
void criafila(FILAENC *f) {
    f->INICIO = NULL;
    f->FIM = NULL;
}
```

2. Inserir elemento:

```
void ins(FILAENC *f, int v) {
    NODO *novo = (NODO*)malloc(sizeof(NODO));
    if (!novo) {
        printf("Memória insuficiente!");
        exit(1);
    }

    novo->info = v;
    novo->prox = NULL;

    if (f->FIM == NULL) { // Fila vazia
        f->INICIO = novo;
        f->FIM = novo;
    } else { // Fila não vazia
        f->FIM->prox = novo;
        f->FIM = novo;
    }
}
```

Explicação: Duas situações: fila vazia (novo nó é início e fim) ou fila não vazia (anexa ao final).

3. Remover elemento:

```
void ret(FILAENC *f) {
    if (f->INICIO == NULL) {
        printf("Fila vazia!");
        exit(1);
    }

    NODO *aux = f->INICIO;
    f->INICIO = f->INICIO->prox;

    if (f->INICIO == NULL) { // Fila ficou vazia
        f->FIM = NULL;
    }

    free(aux);
}
```

▮ Videoaula 20 - Aplicação de Filas - Ordenação por Distribuição (19 min)

Link: https://www.youtube.com/watch?v=MdFgLwcmqS4&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=20

II. PILHAS (Stack) - LIFO (Last In, First Out)

2.1 Conceitos Fundamentais

Como o Professor Linder explica: "Uma pilha é uma lista com restrições de acesso, onde todas as operações só podem ser aplicadas sobre o topo. Estabelece-se o critério LIFO - Last In, First Out."

Operações Básicas:

- **Push (Empilhar):** inserção no topo da pilha
- **Pop (Desempilhar):** remoção do topo da pilha
- **Top/Peak:** acesso ao elemento do topo sem removê-lo
- **isEmpty:** verificação se a pilha está vazia

2.2 Implementação Estática Sequencial

▮ Videoaula 21 - Pilha - Alocação Sequencial (18 min)

Link: https://www.youtube.com/watch?v=ZakKeMjn5Q8&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=21

Estrutura de Dados:

```
#define MAX 100

typedef struct {
    int VAL[MAX];    // Vetor para armazenar elementos
    int TOPO;        // Cursor para topo da pilha
} PILHASEQ;
```

Explicação do Professor: "A pilha é mais simples que a fila - precisamos apenas de um cursor TOPO que indica onde está o topo da pilha."

Operações Implementadas:

1. Criar Pilha:

```
void criapilha(PILHASEQ *p) {
    p->TOPO = -1; // Pilha vazia
}
```

Explicação: TOPO = -1 indica pilha vazia. Quando inserimos o primeiro elemento, TOPO vira 0.

2. Verificar se está vazia:

```
int ehvazia(PILHASEQ *p) {
    return (p->TOPO == -1);
}
```

3. Inserir elemento (Push):

```
void push(PILHASEQ *p, int v) {
    if (p->TOPO == MAX - 1) {
```

```

        printf("Pilha cheia!");
        exit(1);
    }
    p->TOP0++;
    p->VAL[p->TOP0] = v;
}

```

Explicação: Incrementa TOPO primeiro, depois insere o valor na nova posição.

4. Consultar topo:

```

int top(PILHASEQ *p) {
    if (ehvazia(p)) {
        printf("Pilha vazia!");
        exit(1);
    }
    return p->VAL[p->TOP0];
}

```

5. Remover elemento (Pop):

```

void pop(PILHASEQ *p) {
    if (ehvazia(p)) {
        printf("Pilha vazia!");
        exit(1);
    }
    p->TOP0--;
}

```

Explicação: Simplesmente decrementa TOPO, "esquecendo" o elemento do topo.

2.3 Implementação Dinâmica Encadeada

▮ Videoaula 22 - Pilha Encadeada (14 min)

Link: https://www.youtube.com/watch?v=MJ82dyLEtbU&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=22

Estruturas de Dados:

```

typedef struct nodo {
    int info;
    struct nodo* prox;
} NODO;

typedef NODO* PILHAENC; // Ponteiro para primeiro nó

```

Explicação do Professor: "Como todas as operações ocorrem numa extremidade, a representação se reduz a um único ponteiro para o primeiro nodo."

Operações Implementadas:

1. Criar Pilha:

```
void create(PILHAENC *p) {  
    *p = NULL;  
}
```

2. Inserir elemento (Push):

```
void push(PILHAENC *p, int v) {  
    NODO *novo = (NODO*)malloc(sizeof(NODO));  
    if (!novo) {  
        printf("Memória insuficiente!");  
        exit(1);  
    }  
  
    novo->info = v;  
    novo->prox = *p; // Novo nó aponta para antigo topo  
    *p = novo;      // Pilha aponta para novo nó  
}
```

Explicação: O novo nó sempre se torna o novo topo, apontando para o antigo topo.

3. Remover elemento (Pop):

```
void pop(PILHAENC *p) {  
    if (*p == NULL) {  
        printf("Pilha vazia!");  
        exit(1);  
    }  
  
    NODO *aux = *p;  
    *p = (*p)->prox; // Pilha aponta para segundo nó  
    free(aux);       // Libera memória do antigo topo  
}
```

▮ Videoaula 23 - Aplicação de Pilhas - Expressões Aritméticas (53 min)

Link: https://www.youtube.com/watch?v=bXqNU-4SU1I&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=23

III. ÁRVORES - Estruturas Hierárquicas

3.1 Conceitos Fundamentais

▮ Videoaula 24 - Árvores - Caracterização - Árvores Binárias (14 min)

Link: https://www.youtube.com/watch?v=9W8Nit4PHac&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=24

Como o Professor Linder explica: "Pilhas e filas são limitadas a apenas uma dimensão. Para representação hierárquica, criou-se o conceito de árvore."

Terminologia Essencial:

- **Nó/Vértice:** cada elemento da árvore
- **Raiz:** nó principal (nível 0)
- **Folha:** nó sem filhos
- **Altura/Profundidade:** nível máximo da árvore
- **Caminho:** sequência de arcos da raiz até um nó
- **Árvore Binária:** cada nó tem no máximo 2 filhos

Propriedade importante: Em uma árvore binária, existem no máximo 2^i nós no nível i .

3.2 Implementação Estática Sequencial

▮ Videoaula 25 - Árvore Binária - Armazenamento Estático (31 min)

Link: https://www.youtube.com/watch?v=akfMDItMoJw&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=25

Estruturas de Dados:

```
#define MAX 100

typedef struct {
    int info;        // Informação do nó
    int left;        // Índice do filho esquerdo
    int right;       // Índice do filho direito
    int father;      // Índice do pai
} NODO;

typedef struct {
    NODO nodos[MAX];
    int raiz;        // Índice da raiz
    int nodoLivre;    // Primeiro nó livre
} ARVBINSEQ;
```

Explicação do Professor: "No armazenamento estático, usamos índices para referenciar outros nós. -1 indica referência inválida."

Operações Básicas:

1. Criar Árvore:

```
void maketree(ARVBINSEQ *t, int x) {
    if (t->nodoLivre == -1) {
        printf("Árvore cheia!");
    }
```



```

        exit(1);
    }

    int p = t-&t;nodoLivre;
    t-&t;nodoLivre = t-&t;nodos[p].right; // Próximo livre

    t-&t;nodos[p].info = x;
    t-&t;nodos[p].left = -1;
    t-&t;nodos[p].right = -1;
    t-&t;nodos[p].father = -1;

    t-&t;raiz = p;
}

```

2. Inserir filho esquerdo:

```

void setleft(ARVBINSEQ *t, int p, int x) {
    if (t-&t;nodos[p].left != -1) {
        printf("Já existe filho esquerdo!");
        exit(1);
    }

    int q = t-&t;nodoLivre;
    t-&t;nodoLivre = t-&t;nodos[q].right;

    t-&t;nodos[q].info = x;
    t-&t;nodos[q].left = -1;
    t-&t;nodos[q].right = -1;
    t-&t;nodos[q].father = p;

    t-&t;nodos[p].left = q;
}

```

3.3 Implementação Dinâmica Encadeada

▮ Videoaula 26 - Árvore Binária - Armazenamento Dinâmico (26 min)

Link: https://www.youtube.com/watch?v=UZGWEdkmNkY&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=26

Estruturas de Dados:

```

typedef struct nodo {
    int info;
    struct nodo* esquerdo;
    struct nodo* direito;
} NODO;

typedef NODO* ARVBINENC;

```

Explicação do Professor: "O armazenamento dinâmico otimiza o uso da memória, alocando apenas os nós necessários."

Operações com Recursão:

1. Percurso Em Ordem (In-Order):

```
void preordem(ARVBINENC raiz) {
    if (raiz != NULL) {
        printf("%d ", raiz->info);           // Visita raiz
        preordem(raiz->esquerdo);           // Percorre esquerda
        preordem(raiz->direito);             // Percorre direita
    }
}
```

2. Percurso Em Ordem (In-Order):

```
void emordem(ARVBINENC raiz) {
    if (raiz != NULL) {
        emordem(raiz->esquerdo);           // Percorre esquerda
        printf("%d ", raiz->info);         // Visita raiz
        emordem(raiz->direito);            // Percorre direita
    }
}
```

3. Percurso Pós-Ordem (Post-Order):

```
void posordem(ARVBINENC raiz) {
    if (raiz != NULL) {
        posordem(raiz->esquerdo);           // Percorre esquerda
        posordem(raiz->direito);            // Percorre direita
        printf("%d ", raiz->info);         // Visita raiz
    }
}
```

3.4 Árvores Binárias de Busca

▮ Videoaula 27 - Árvore Binária de Busca (32 min)

Link: https://www.youtube.com/watch?v=1Vm-82q8zg8&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=27

Como o Professor Linder explica: "Uma árvore binária de busca tem a propriedade fundamental: para cada nó, todos os valores da subárvore esquerda são menores, e todos da subárvore direita são maiores ou iguais."

Operações Implementadas:

1. Buscar elemento:

```
NODO* buscar(ARVBINENC raiz, int x) {
    if (raiz == NULL || raiz->info == x) {
        return raiz;
    }

    if (x < raiz->info) {
```

```

        return buscar(raiz->esquerdo, x); // Busca à esquerda
    } else {
        return buscar(raiz->direito, x); // Busca à direita
    }
}

```

2. Inserir elemento:

```

void inserir(ARVBINENC *raiz, int x) {
    if (*raiz == NULL) {
        *raiz = (NODO*)malloc(sizeof(NODO));
        (*raiz)->info = x;
        (*raiz)->esquerdo = NULL;
        (*raiz)->direito = NULL;
    } else if (x < (*raiz)->info) {
        inserir(&(*raiz)->esquerdo, x);
    } else {
        inserir(&(*raiz)->direito, x);
    }
}

```

3. Remover elemento (casos complexos):

Caso 1: Nó folha (sem filhos)

```

// Simplesmente remove o nó
if (no->esquerdo == NULL && no->direito == NULL) {
    free(no);
    *raiz = NULL;
}

```

Caso 2: Nó com um filho

```

// Substitui o nó pelo seu único filho
if (no->esquerdo == NULL) {
    NODO *temp = no;
    *raiz = no->direito;
    free(temp);
}

```

Caso 3: Nó com dois filhos

- **Remoção por Fuso:** anexa uma subárvore à outra
- **Remoção por Cópia:** substitui pelo sucessor imediato (menor da subárvore direita)

3.5 Árvores Balanceadas - AVL

▮ Videoaula 28 - Árvore Balanceada - Árvore AVL (44 min)

Link: https://www.youtube.com/watch?v=NSBA4mVe4xM&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=28

Como o Professor Linder explica: "Uma árvore AVL é uma árvore binária de busca onde a diferença em altura entre as subárvores esquerda e direita de cada nó é no máximo 1."

Fator de Balanceamento:

$$FB(p) = altura(sub\acute{a}rvore_direita) - altura(sub\acute{a}rvore_esquerda)$$

Estrutura de Dados:

```
typedef struct nodo_avl {
    int info;
    int altura;                // Campo adicional para altura
    struct nodo_avl* esquerdo;
    struct nodo_avl* direito;
} NOAVL;

typedef NOAVL* ARVAVL;
```

Rotações para Balanceamento:

1. Rotação Direita (Simples):

```
NOAVL* rotacaoDireita(NOAVL* y) {
    NOAVL* x = y->esquerdo;
    NOAVL* T2 = x->direito;

    // Rotação
    x->direito = y;
    y->esquerdo = T2;

    // Atualizar alturas
    y->altura = max(altura(y->esquerdo), altura(y->direito)) + 1;
    x->altura = max(altura(x->esquerdo), altura(x->direito)) + 1;

    return x; // Nova raiz
}
```

2. Rotação Esquerda (Simples):

```
NOAVL* rotacaoEsquerda(NOAVL* x) {
    NOAVL* y = x->direito;
    NOAVL* T2 = y->esquerdo;

    // Rotação
    y->esquerdo = x;
    x->direito = T2;

    // Atualizar alturas
    x->altura = max(altura(x->esquerdo), altura(x->direito)) + 1;
```

```

y->altura = max(altura(y->esquerdo), altura(y->direito)) + 1;

return y; // Nova raiz
}

```

Classes de Desbalanceamento:

- **Classe 1:** FB pai e filho mesmo sinal → Rotação simples
- **Classe 2:** FB pai e filho sinais opostos → Rotação dupla

IV. CRONOGRAMA DE ESTUDO DETALHADO

Semana 1 (7-13 de outubro) - Filas e Pilhas

Dia	Atividade	Duração	Link Direto	Observações
Segunda (7/10)	Videoaula 18 - Filas Estáticas	32 min	https://youtu.be/vZbFTtr--QU	Foque nos cursores INICIO/FIM
	Implementar TAD FILASEQ	45 min	-	Todas as operações básicas
Terça (8/10)	Videoaula 19 - Fila Encadeada	27 min	https://youtu.be/Gq9V6wzyrVY	Atenção ao descritor
	Implementar TAD FILAENC	45 min	-	Cuidado com malloc/free
Quarta (9/10)	Videoaula 21 - Pilha Sequencial	18 min	https://youtu.be/ZakKeMjn5Q8	Conceito de TOPO
	Implementar TAD PILHASEQ	30 min	-	Mais simples que filas
Quinta (10/10)	Videoaula 22 - Pilha Encadeada	14 min	https://youtu.be/MJ82dyLEtbU	Apenas um ponteiro
	Implementar TAD PILHAENC	30 min	-	Inserção no início
Sexta (11/10)	Videoaula 20 - Aplicações de Filas	19 min	https://youtu.be/MdFgLwcmqS4	Algoritmos práticos
	Revisão Completa Filas/Pilhas	60 min	-	Comparar implementações

Segunda Semana (14-15 de outubro) - Árvores

Dia	Atividade	Duração	Link Direto	Observações
Segunda (14/10)	Videoaula 24 - Conceitos de Árvores	14 min	https://youtu.be/9W8Nit4PHac	Terminologia fundamental
	Videoaula 25 - Armazenamento Estático	31 min	https://youtu.be/akfMDItMoJw	Índices como ponteiros

Dia	Atividade	Duração	Link Direto	Observações
	Implementar operações básicas	45 min	-	maketree, setleft, setright
Terça (15/10)	Videoaula 26 - Armazenamento Dinâmico	26 min	https://youtu.be/UZGWEdkmNkY	Ponteiros para nós
	Videoaula 27 - Árvore de Busca	32 min	https://youtu.be/1Vm-82q8zg8	Propriedade fundamental
	Videoaula 28 - Árvore AVL	44 min	https://youtu.be/NSBA4mVe4xM	Rotações e balanceamento
	SEGUNDA PROVA	-	-	Boa sorte!

V. PONTOS CRÍTICOS PARA A PROVA

5.1 Diferenças Entre Implementações

Aspecto	Estática	Dinâmica
Memória	Fixa (pode desperdiçar)	Flexível (apenas necessária)
Complexidade	Mais simples	Requer malloc/free
Performance	Acesso $O(1)$	Acesso $O(1)$ + overhead
Flexibilidade	Limitada pelo MAX	Limitada pela memória

5.2 Armadilhas Comuns

1. **Filas Circulares:** Lembrar do operador % MAX
2. **Casos Extremos:** Estruturas vazias, inserção quando cheia
3. **Ponteiros Nulos:** Sempre verificar antes de desreferenciar
4. **Liberação de Memória:** Todo malloc precisa de free correspondente
5. **Árvores AVL:** Diferenciar rotação simples de dupla

5.3 Complexidade das Operações

Estrutura	Inserção	Remoção	Consulta	Busca
Fila	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Pilha	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Árvore Binária	$O(n)$	$O(n)$	-	$O(n)$
Árvore de Busca	$O(h)$	$O(h)$	-	$O(h)$
Árvore AVL	$O(\log n)$	$O(\log n)$	-	$O(\log n)$

onde h = altura da árvore

VI. EXERCÍCIOS DE FIXAÇÃO

6.1 Filas

1. Implemente uma fila usando duas pilhas
2. Crie uma função que inverta uma fila usando uma pilha
3. Simule um sistema de atendimento com prioridades

6.2 Pilhas

1. Implemente uma calculadora pós-fixa
2. Verifique se parênteses estão balanceados
3. Converta expressão infixa para pós-fixa

6.3 Árvores

1. Conte o número de nós folhas
2. Calcule a altura de uma árvore
3. Implemente busca em largura usando fila
4. Determine se uma árvore é AVL

VII. LINKS RÁPIDOS - LISTA COMPLETA

Filas (Queue) - ESSENCIAL ★★★

1. **Videoaula 18 - Disciplinas de Acesso - Fila** (32 min)
▯ https://www.youtube.com/watch?v=vZbFTtr--QU&list=PLMx6FZj7-18bH9UWzsY31zf45eHnOI_dO&index=18
2. **Videoaula 19 - Fila Encadeada** (27 min)
▯ https://www.youtube.com/watch?v=Gq9V6wzyrVY&list=PLMx6FZj7-18bH9UWzsY31zf45eHnOI_dO&index=19
3. **Videoaula 20 - Exemplo de Aplicação de Filas** (19 min)
▯ https://www.youtube.com/watch?v=MdFgLwcmqS4&list=PLMx6FZj7-18bH9UWzsY31zf45eHnOI_dO&index=20

Pilhas (Stack) - ESSENCIAL ★★★

4. **Videoaula 21 - Pilha - Alocação Sequencial** (18 min)
▯ https://www.youtube.com/watch?v=ZakKeMjn5Q8&list=PLMx6FZj7-18bH9UWzsY31zf45eHnOI_dO&index=21
5. **Videoaula 22 - Pilha Encadeada** (14 min)
▯ https://www.youtube.com/watch?v=MJ82dyLEtbU&list=PLMx6FZj7-18bH9UWzsY31zf45eHnOI_dO&index=22

[O&index=22](#)

6. Videoaula 23 - Aplicação de Pilhas - Expressões (53 min)

▯ https://www.youtube.com/watch?v=bXqNU-4SU1l&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=23

Árvores (Trees) - ESSENCIAL ★★★

7. Videoaula 24 - Árvores - Caracterização (14 min)

▯ https://www.youtube.com/watch?v=9W8Nit4PHac&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=24

8. Videoaula 25 - Árvore Binária - Armazenamento Estático (31 min)

▯ https://www.youtube.com/watch?v=akfMDltMoJw&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=25

9. Videoaula 26 - Árvore Binária - Armazenamento Dinâmico (26 min)

▯ https://www.youtube.com/watch?v=UZGWEdkmNkY&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=26

10. Videoaula 27 - Árvore Binária de Busca (32 min)

▯ https://www.youtube.com/watch?v=1Vm-82q8zg8&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=27

11. Videoaula 28 - Árvore Balanceada - Árvore AVL (44 min)

▯ https://www.youtube.com/watch?v=NSBA4mVe4xM&list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO&index=28

▯ Playlist Completa:

https://youtube.com/playlist?list=PLMx6FZj7-18bH9UWzsY31zf45eHn0I_dO

VIII. RESUMO FINAL

✓ Checklist de Estudo:

- ☐ Entender FIFO (filas) vs LIFO (pilhas)
- ☐ Dominar implementações estáticas vs dinâmicas
- ☐ Conhecer todas as operações básicas
- ☐ Saber quando usar cada estrutura
- ☐ Compreender árvores binárias de busca
- ☐ Entender balanceamento AVL e rotações
- ☐ Praticar implementação em C
- ☐ Revisar complexidade de tempo e espaço

▮ Distribuição do Tempo:

- **Vídeos de Alta Prioridade:** 4h17min (9 vídeos)
- **Vídeos Complementares:** 1h12min (2 vídeos)
- **Total de Conteúdo:** 5h09min (11 vídeos)

▮ Foco Principal:

1. **Filas:** Entender fila circular e implementação dinâmica
2. **Pilhas:** Dominar aplicações (expressões, recursão)
3. **Árvores:** Percursos, busca, inserção e remoção
4. **AVL:** Rotações simples e duplas

Mensagem Final: "A prova testará sua capacidade de implementar e aplicar estas estruturas. Pratique os códigos, entenda os conceitos e confie no seu preparo!"

Boa sorte na segunda prova! ▮

"O sucesso é a soma de pequenos esforços repetidos dia após dia." - Robert Collier